Practice Questions

Question1. Assume that the following processes are the only processes in a computer system. Given each process's arrival time and CPU burst time, answer the following questions.

Process	Arrival Time	CPU Burst Time
P_1	0	20
P_2	3	7
P_3	5	2
P_4	7	3

Draw the Gantt chart and compute the average waiting time for the following CPU scheduling algorithms.

- First Come First Service (FCFS).
- Preemptive Shortest Job Next (also called Shortest Remaining Job Next).
- Round-Robin with time quantum of 4 (Assume that new processes will be inserted at the end of the ready list).

Question 2: Assume you have a system with three processes (X, Y, and Z) and a single CPU. Process X has the highest priority, process Z has the lowest priority, and Y is in the middle. Assume a priority-based scheduler (i.e., the scheduler runs the highest priority job, performing preemption as necessary). Processes can be in one of five states: RUNNING, READY, BLOCKED, not yet created, or terminated.
Given the following cumulative timeline of process behavior, indicate the state the specified process is in AFTER that step and all preceding steps have taken place. Assume the scheduler has reacted to the specified workload change. For all questions in this part, use the following options for each answer:

a. RUNNING
b. READY
c. BLOCKED
d. Process has not been created yet
e. Not enough information to determine OR None of the above

Step 1:
Process X is loaded into memory and begins; it is the only user-level process in the system.
1) Process X is in which state?

Step 2:
Process X calls fork() and creates Process Y.
2) Process X is in which state?
3) Process Y is in which state?

Step 3:
The running process issues an I/O request to the disk.
4) Process X is in which state?
5) Process Y is in which state?

Step 4:
The running process calls fork() and creates Process Z.
6) Process X is in which state?
7) Process Y is in which state?
8) Process Z is in which state?

Step 5:
The previously issued I/O request completes.
9) Process X is in which state?
10) Process Y is in which state?
11) Process Z is in which state?

Step 6:
The running process completes.
12) Process X is in which state?
13) Process Y is in which state?
14) Process Z is in which state?

Question 3. Answer the following two questions based on the code provided below. Assume the code has been compiled and executed.

```
main() {
   int a = 0;
   int rc = fork();
   a++;
   if (rc == 0) {
         rc = fork();
         a++;
   } else {
         a++;
   }
   printf("Hello!\n");
   printf("a is %d\n", a);
}
```

A. Assuming fork() never fails, how many times will the message "Hello!\n" be displayed?
   a. 2
   b. 3
   c. 4
   d. 6
   e. None of the above

B. What will be the largest value of "a" displayed by the program?
   a. Due to race conditions, "a" may have different values on different runs of the program.
   b. 2
   c. 3
   d. 5
   e. None of the above

Question 4. Answer the following two questions based on the code provided below. Assume the code has been compiled and executed.

```c
#include <stdio.h>
#include <pthread.h>

int balance = 0;

void *mythread(void *arg) {
    int i;
    for (i = 0; i < 200; i++) {
        balance++;
    }
    printf("Balance is %d\n", balance);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p1, p2, p3;

    pthread_create(&p1, NULL, mythread, "A");
    pthread_join(p1, NULL);

    pthread_create(&p2, NULL, mythread, "B");
    pthread_join(p2, NULL);

    pthread_create(&p3, NULL, mythread, "C");
    pthread_join(p3, NULL);

    printf("Final Balance is %d\n", balance);

    return 0;
}
```

A. Assuming none of the system calls fail, when thread p1 prints "Balance is %d\n", what will p1 say is the value of balance?

a) Due to race conditions, "balance" may have different values on different runs of the program.

b) 200

c) 400

d) 600

e) None of the above

B. Assuming none of the system calls fail, when the main (parent) thread prints "Final Balance is %d\n", what will the parent thread say is the value of balance?

a) Due to race conditions, "balance" may have different values on different runs of the program.

b) 200

c) 400

d) 600

e) None of the above

Question 5. One approach for using compare and swap() for implementing a spinlock is as follows:

```
void lock_spinlock(int *lock) {
   while (compare_and_swap(lock, 0, 1) != 0)
      ; /* spin */
}
```

A suggested alternative approach is to use the “compare and compare-and-swap” idiom, which checks the lock's status before invoking the compare and swap() operation. (The rationale behind this approach is to invoke compare and swap() only if the lock is currently available.) This strategy is shown below:

```
void lock_spinlock(int *lock) {
{
   while (true) {
      if (*lock == 0) {
         /* lock appears to be available */

         if (!compare_and_swap(lock, 0, 1))
            break;
      }
   }
}
```

Does this “compare and compare-and-swap” idiom work appropriately for implementing spinlocks? If so, explain. If not, illustrate how the integrity of the lock is compromised.

Question 6. Which of the following scheduling algorithms could result in starvation? Provide a brief justification for your answer.

a. First-come, first-served
b. Shortest job first
c. Round robin
d. Priority

Question 7. Two threads, T1 and T2, are executing the following code segments. The goal is to ensure that T1 and T2 alternate their execution indefinitely, with T1 always executing first. Use two semaphores, sem1 and sem2, to enforce this synchronization. Additionally, the semaphores must prevent busy-waiting and ensure that each thread only runs after the other has completed its turn.

```
Thread T1:
    wait(sem1);
    print("T1 is running");
    signal(sem2);

Thread T2:
    wait(sem2);
    print("T2 is running");
    signal(sem1);
```

Questions:

1. What should the initial values of sem1 and sem2 be, and why?
2. Explain how the two semaphores (sem1 and sem2) ensure that T1 and T2 alternate their execution in a synchronized manner.
3. Modify the program so that three threads, T1, T2, and T3, execute in alternating order (T1, T2, T3, T1, T2, T3, ...).

All the best!